

Music Scripting Engine User Manual

Setup and Installation of the Codebase.....	3
Required Dependencies.....	3
Optional dependencies.....	4
Your First Composition.....	4
Working with Notes.....	4
Three Ways to Specify Pitch.....	4
Velocity & Silence.....	5
Duration Values.....	5
Sequences & Algorithms.....	5
Multiple Sequences.....	5
Random Walk (Generate Melodies).....	5
Rhythm Generator.....	6
Arpeggio Generator.....	6
Multi-Track Compositions.....	6
Exporting & Playback.....	7
Export to MIDI.....	7
Real-Time Playback.....	7
Control Playback.....	7
Quick Reference.....	7
Common Commands.....	7
Common Note Names.....	7
Troubleshooting.....	8
No sound plays?.....	8
'ModuleNotFoundError: No module named 'music_engine'.....	8
Wrong pitches?.....	8
Composition plays too fast/slow?.....	8
Wrong notes in exported MIDI?.....	8
Tips.....	9
1. Use loops for repetition:.....	9
2. Use variables for key pitches:.....	9
3. Crescendo effect:.....	9
4. Use moderate tempos: 60-240 BPM works best.....	9
Examples.....	9
Mary Had a Little Lamb.....	9
Two-Part Composition.....	10
FAQ.....	10
Q: Can I use different instruments?.....	10
Q: Can I change waveforms (sine, square, triangle)?.....	10
Q: How long can compositions be?.....	10
Q: Can I use swing or triplets?.....	10
Q: What about time signatures (5/4, 7/8)?.....	11
Next Steps.....	11

This is a user manual for the learning and understanding of the Python music scripting engine. For further information on maintenance, please refer to the maintenance manual and relevant examples within the codebase.

Setup and Installation of the Codebase

Once the codebase is downloaded from GitHub, unzip the folder and open the folder as a workspace within your preferred IDE (e.g. VSCode or PyCharm).

The root file structure should look as follows:

```
|— music_engine/          # Main package
| |— __init__.py
| |— algorithms/         # Generative algorithms
| | |— arpeggio.py
| | |— random_walk.py
| | |— rhythm.py
| |— core/              # Core musical objects
| | |— chord.py
| | |— note.py
| | |— sequence.py
| | |— track.py
| | |— __init__.py
| |— engine/            # Audio engine
| | |— controller.py
| | |— __init__.py
| |— output/            # Export & playback
| | |— midi_export.py
| | |— playback.py
| | |— __init__.py
| |— tests/             # Test suite
| | |— test_*.py        # Individual module tests
| | |— __init__.py
|— runner.py            # High-level API interface
|— ex[1-6]_*.py         # Example scripts
|— task[1-3].py         # Assignment tasks
|— MAINTENANCE_MANUAL.md # This file
|— output.mid           # Generated MIDI (if any)
```

Required Dependencies

Before running the codebase, it is important that the correct prerequisites/dependencies are installed properly.

- Python 3.7 or higher must be used.
- pip package manager must be used for installing the relevant dependencies. More information on pip can be found at the following address:

<https://packaging.python.org/en/latest/tutorials/installing-packages/>

Enter the command 'pip install mido numpy sounddevice' into the terminal to install the required packages.

Optional dependencies

For running the built-in module tests pytest can be used. Use the command 'pip install pytest' to install its package. More information on pytest can be found at the following address:

<https://docs.pytest.org/>

Your First Composition

Create 'my_song.py' in the root directory and copy the following code into the file and save it:

```
from runner import *

@register_track
def my_song(piano):
    melody = piano.add_sequence("verse")
    melody.add_note('C4', 1.0)
    melody.add_note('D4', 1.0)
    melody.add_note('E4', 1.0)
    melody.add_note('C4', 1.0)
    melody.play(tempo=120)

my_song(TrackProxy("Piano"))
```

Next run this file and observe the output, using 'python my_song.py' in the terminal, or by pressing play in your IDE.

Working with Notes

Three Ways to Specify Pitch

```
seq.add_note('C4', 1.0)           # Note name (intuitive)
```

```
seq.add_note(60, 1.0)          # MIDI number (Middle C)
seq.add_note(261.63, 1.0)      # Frequency in Hz
```

Note format: '[A-G][#/b?][octave]' e.g. C4, D#4, Bb3, F5
 Octave: 4=middle, 3=lower, 5=higher

Velocity & Silence

```
seq.add_note('C4', 1.0, velocity=80) # Volume (0-100, default 64)
seq.wait(1.0)                        # Add silence
```

Duration Values

0.25 = sixteenth note
 0.5 = eighth
 1.0 = quarter
 2.0 = half note
 4.0 = whole note

Sequences & Algorithms

Multiple Sequences

```
@register_track
def song(piano):
    intro = piano.add_sequence("intro")
    intro.add_note('C4', 1.0)

    verse = piano.add_sequence("verse")
    verse.add_note('D4', 1.0)
    verse.add_note('E4', 1.0)

    intro.play(tempo=120)
    verse.play(tempo=120)

song(TrackProxy("Piano"))
```

Random Walk (Generate Melodies)

```
seq.generate_random_walk(
    num_notes=16,
    scale='major_c',          # 'minor_a', 'chromatic'
    direction='up',           # 'down', 'random'
```

```
    start_pitch='C4'  
)
```

Rhythm Generator

```
seq.generate_rhythm_sequence(  
    num_notes=8,  
    beat_target=4.0          # Fill exactly 4 beats  
)
```

Arpeggio Generator

```
seq.generate_arpeggio(  
    root_pitch='C4',  
    chord_type='major',      # 'minor', 'seventh', 'diminished', 'augmented'  
    num_notes=16,  
    direction='up'  
)
```

Multi-Track Compositions

```
from runner import *  
  
@register_track  
def melody_part(piano):  
    seq = piano.add_sequence("tune")  
    seq.add_note('C4', 4.0)  
    seq.play(tempo=120)  
  
@register_track  
def bass_part(bass):  
    seq = bass.add_sequence("bass")  
    seq.add_note('C1', 4.0)  
    seq.play(tempo=120)  
  
melody_part(TrackProxy("Piano"))  
bass_part(TrackProxy("Bass"))  
  
play_polyphonic() # Play both simultaneously
```

Exporting & Playback

Export to MIDI

```
export_midi("my_song.mid", tempo=120)
```

Use in any DAW: Ableton Live, Logic Pro, FL Studio, GarageBand, etc.

Real-Time Playback

```
seq.play(tempo=120)           # Play single sequence
play_polyphonic()            # Play all tracks together
```

Control Playback

```
seq.play(tempo=60)           # Slower
seq.play(tempo=120)          # Normal
seq.play(tempo=240)          # Faster
```

Quick Reference

Common Commands

```
# Adding notes
sequence.add_note('C4', 1.0)           # Simple
sequence.add_note('C4', 1.0, velocity=80) # With volume
sequence.wait(1.0)                     # Silence

# Playback & export
sequence.play(tempo=120)
export_midi("song.mid", tempo=120)
play_polyphonic()

# Algorithms
seq.generate_random_walk(16, 'major_c', 'up', 'C4')
seq.generate_rhythm_sequence(8, 4.0)
seq.generate_arpeggio('C4', 'major', 16, 'up')
```

Common Note Names

Note	Frequency	MIDI
C4 (Middle C)	261.63 Hz	60
A4 (Concert Pitch)	440.00 Hz	69
C3 (Lower)	130.81 Hz	48
C5 (Higher)	523.25 Hz	72

Troubleshooting

No sound plays?

- Check system volume and speaker connection
- Verify velocity isn't 0: `'velocity=80'`
- Check sequence has notes: `'print(len(seq.get_notes()))'`

'ModuleNotFoundError: No module named 'music_engine''

- Ensure you're in the project directory
- Run with: `'python your_script.py'` from the correct folder

Wrong pitches?

- Verify note names: `'C4'` not `'C'` or `'C4#'`
- Octave: 4=middle, 3=lower, 5=higher
- Test: `'Note('C4', 1).get_midi_pitch()'` should print 60

Composition plays too fast/slow?

- Adjust tempo: `'seq.play(tempo=80)'` or `'tempo=160'`

Wrong notes in exported MIDI?

- Export with matching tempo: `'export_midi("song.mid", tempo=120)'`

Tips

1. Use loops for repetition:

```
pattern = ['C4', 'D4', 'E4']
for note in pattern * 4:
    seq.add_note(note, 0.5)
```

2. Use variables for key pitches:

```
root, third, fifth = 'C4', 'E4', 'G4'
seq.add_note(root, 1.0)
seq.add_note(third, 1.0)
```

3. Crescendo effect:

```
for i, pitch in enumerate(['C4', 'D4', 'E4', 'F4']):
    velocity = 30 + (i * 20)
    seq.add_note(pitch, 1.0, velocity=velocity)
```

4. Use moderate tempos: 60-240 BPM works best

Examples

Mary Had a Little Lamb

```
from runner import *

@register_track
def mary(piano):
    m = piano.add_sequence("tune")
    notes = 'E D C D E E E D D E G'.split()
    durs = [0.5]*3 + [0.5, 0.5, 0.5, 0.5, 0.5] + [0.5, 0.5, 0.5, 0.5, 1.0]
    for note, dur in zip(notes, durs):
        m.add_note(note + '4', dur)
    m.play(tempo=120)

mary(TrackProxy("Piano"))
```

Two-Part Composition

```
from runner import *

@register_track
def melody(piano):
    top = piano.add_sequence("melody")
    top.generate_random_walk(16, 'major_c', 'up', 'C4')
    top.play(tempo=120)

@register_track
def bass(bass_guitar):
    bottom = bass_guitar.add_sequence("bass")
    bottom.add_note('C2', 4.0)
    bottom.add_note('F1', 4.0)
    bottom.play(tempo=120)

melody(TrackProxy("Piano"))
bass(TrackProxy("Bass Guitar"))
play_polyphonic()
```

FAQ

Q: Can I use different instruments?

A: The Python API uses generic names. Export to MIDI to assign different instruments in a DAW.

Q: Can I change waveforms (sine, square, triangle)?

A: Currently, sine wave only. Export to MIDI for different sounds in a DAW.

Q: How long can compositions be?

A: Unlimited. Length is limited only by RAM and patience.

Q: Can I use swing or triplets?

A: Use fractional durations (0.333, 0.667) or modify sequences manually.

Q: What about time signatures (5/4, 7/8)?

A: No built-in support. Manually create sequences or use `beat_target` in rhythm generation.

Next Steps

1. Run the examples: 'python ex1_creating_scripts.py', etc.
2. Modify an example and experiment
3. Create your own composition
4. Export to MIDI and use in your favourite DAW
5. Check MAINTENANCE_MANUAL.md for advanced topics

****For more details, see MAINTENANCE_MANUAL.pdf****